MAE544 Project Final Report

Jacob Derby
University at Buffalo, SUNY

May 15, 2026

Code Repository

Comparative Analysis of Iterative Learning Control (ILC) and Model Reference Adaptive Control (MRAC) for a Discrete-Time Stepper Motor

Jacob Derby

A. ABSTRACT

Modern day manufacturing contains a multitude of nonlinear discrete-time systems such as stepper motors that conduct the same precise and repeated movements over a finite amount of time but suffer from unmodeled dynamics such as discontinuous friction or varying loads. Standard control methods such as Proportional Integral Derivative (PID) tend to lack the capability to eliminate residual tracking error entirely across the successive iterations these systems see. In this study, a NEMA 23 Stepper Motor tracking a 1 Hz sinusoidal trajectory was designed and used to benchmark three discrete-time control methods against each other. These three control architectures include a baseline PID, batch-learning ILC, and a real-time MRAC. The study irrefutably demonstrates the ILC's ability to outperform other control methods in repeated-trajectory systems such as this, nearly eliminating tracking error with minimal control effort. The implementation of MRAC demonstrated its capability of tracking an ideal model without prior learning opportunities, but the causality constraints associated with it led to slight phase lag and somewhat erratic control effort as a result of sensor noise.

B. INTRODUCTION

Stepper motors are a ubiquitous technology used in modern manufacturing because of their cheap, precise, and versatile nature. However, the modeling of stepper motors is nontrivial, as they suffer from highly nonlinear and unmodeled dynamics resulting from unanticipated external factors such as discontinuous friction and magnetic detent torque. This lack of linearity is the exact reason why standard linear control methods such as PID fail to eliminate tracking error. So, for an application such as this where the same task is repeatedly completed, ILC is a highly effective solution that allows the controller to learn from past mistakes and improve upon them progressively through each trial, per [1] - [3]. While ILC performs exceptionally under repeated tasks, MRAC offers a highly optimal control technique under unpredictable, continuous disturbances that a system like this would see, per [4].

The objective of this project is to perform a comparative analysis between a baseline PID, a batch-learning ILC, and a real-time MRAC. All these control architectures are tested against a continuous sinusoidal reference trajectory to examine their ability to reduce tracking error and reject unmodeled dynamics or sensor noise.

C. LITERATURE REVIEW

Paper [1]

This thesis offers an extensive overview of the history, motivation, and mathematical method behind model-based ILC for discrete-time systems that serve as the reason it is such a powerful form of control. It details exactly *why* ILC works and lays a stable foundation for a wide variety of the types of systems we would like to examine.

The author establishes the convergence conditions for an ILC system with matrix L and an error evolution defined by $e_{k+1} = Le_k$. The author then compares asymptotic with monotonic convergence, where asymptotic convergence is governed by the spectral radius condition, and monotonic is governed by contraction mapping. Monotonic is the preferred of the two as it guarantees that error will only decrease through each iteration, which is obviously a very desired performance metric for precision applications

Later in the paper, the author discusses time-domain conditions for convergence in ILC with a generic feedforward update law $(u_{k+1} = u_k + \beta K e_k)$. This discussion resultingly proves that if matrix GK only contains eigenvalues in the open right half complex plane and constant gain $\beta > 0$ has a sufficiently small magnitude, the above feedforward ILC algorithm will resultingly converge. The author then explains the two approaches to satisfy the conditions detailed above: the inverse algorithm (where you choose some $K = G^{-1}$) and the adjoint algorithm (choose some $K = G^T$). The inverse algorithm is shown to be capable of theoretically reducing any tracking error to zero, resulting in perfect tracking, but only with an exact system model. While this is not a realistic condition, the author demonstrates that the algorithm can still result in a robust controller that results in monotonic convergence even with model uncertainties. The algorithmic structures provided along with theoretical proofs within the thesis will serve as a direct mathematical foundation for the controller to be designed, simulated, and analyzed within this project.

Paper [2]

This paper builds upon the theory defined in Paper [1] and demonstrates its physical grounding with an application of ILC to a six-axis industrial robot. The authors define a model of the robot's closed-loop dynamics, develop an ILC algorithm to continuously decrease the tracking error even with resonance vibrations and noise, and finally present their experimental results along with analysis and conclusions.

The authors treat the system for this application as Linear Time-Variant (LTV), whereas the proposed stepper motor's system will be modeled as Linear Time-Invariant (LTI). The ILC setup also utilizes the "norm-optimal ILC algorithm", a formulation that minimizes a cost function with weighting matrices V and W, which is a much more complex substitute for the inverse or adjoint algorithms from Paper [1]. A key contribution within this paper is the implementation of a "robustness filter" Q within the input update equation, such that $u_{k+1} = Q(u_k + Le_k)$. This filter is typically used in practical ILC applications as a low-pass filter that reduces the effect of the high-frequency aspect of the input when structural elasticity on the high-frequency dynamics is not accurately modeled. The authors then dive into the modeling of the robot's dynamics, which falls outside of the scope of this project's focus on an LTI system.

Experimental findings show that the robot's tracking error of welding trajectory can be vastly reduced when the ILC is implemented. More specifically, when the cut-off frequency of the Q filter is increased past the first elastic resonance frequency, tracking error is reduced even further. The authors conclude that the accuracy achieved via the ILC was sufficient for these laser welding applications, and that the only remaining error was a result of non-repetitive effects, which cannot be accounted for with ILC. This experiment therefore validates the use of ILCs in these increasingly demanding industrial applications.

Paper [3]

This paper presents a direct application of ILC to this project's hardware focus: a stepper motor. The authors detail the design and implementation of ILC to a "prototype three-phase linear stepper-motor", experimentally validate it, and make concluding remarks regarding its effectiveness in this application. This study is conducted within the frequency domain, which contrasts with the discrete-time domain proposed for this project, but their progression and findings still provide directly applicable insights.

The authors begin with a baseline feedback controller to initially stabilize the system and then proceed to build upon it with ILC feedforward input. They implement methods seen in Papers [1] and [2], including a learning filter L along with a robustness filter Q. The authors note that, although ILC has some practical limitations, these added filters allow it to achieve "… high precision tracking performance to a noise-level-error, exceeding that of the conventional model-based FF control designs". The authors then applied the ILC to the stepper-motor, defining the learning gain α applying it to a plant model $\hat{G}$ that they define within the frequency domain to represent the stepper-motor.

30 trials were then conducted to find the results of applying ILC to this system, finding that even with a prototype setup defined, the scanning operations of the stepper-motor were capable of achieving micrometer levels of precision. These results demonstrate promise for the mitigation of residual error in future stepper-motor systems with more complex setups.

Paper [4]

While [1] – [3] demonstrate that ILC provides optimal tracking capabilities for repetitive tasks, ILC is still inherently vulnerable to unpredictable and non-repeating disturbances. To address this limitation, this paper demonstrated the efficacy of a discrete-time MRAC architecture when applied to an Electronic Throttle Body (ETB), a system that sees many nonlinearities and aerodynamic disturbances that can alter its dynamics.

The authors implemented the MRAC architecture on top of a novel discrete-time Minimal Control Synthesis (MCS) algorithm, known as the Discrete-Time Enhanced MCS (DEMCS). This algorithm effectively resolves the adaptive control problem for discrete-time linear time-invariant (LTI) systems operating in control canonical form (CCF), specifically when plant parameters are unknown and affected by bounded nonlinear disturbances. Through some algebraic manipulation, the authors proved the system is asymptotically hyperstable as long as the feedforward block is positive real and the feedback block satisfied Popov's inequality. The reference model's parameters are then defined based on the hardware's capabilities and driven based on desired performance metrics such as settling time, overshoot, and steady-state tracking error.

The authors tested the DEMCS algorithm on a DV-E5 Bosch ETB installed on a 2 liter naturally aspirated gas engine, in which they demonstrated that the DEMCS algorithm provides superior closed-loop performance than other MRAC algorithms, as it does not suffer from discretization problems. They conclude that the primary advantage of this MRAC approach is its requirement of little knowledge of the plant and its robustness under nonlinearities and persistent aerodynamic perturbations. Resultingly, this specific DEMCS algorithm is adapted into this study to serve as the real-time adaptive control benchmark against the ILC for stepper motor control.

D. SYSTEM MODELING

To properly examine the capabilities of each control architecture, a NEMA 23 hybrid stepper motor was chosen as the physical plant. This specific hardware choice was motivated by [3] as it aligned best with the studied literature and represents an actuator that is ubiquitous in typical manufacturing settings. The theoretical parameters used for the simulation are defined below in Table 1. While these parameters outline the nominal linear system, it is worth noting that the physical motor is still subject to unmodeled dynamics, such as discontinuous friction or detent torque. This therefore justifies the advanced control methodologies employed as necessary for this specific application.

Table 1: NEMA 23 Stepper Motor Parameters

Parameter	Numerical Value
Rotor Inertia	$J = 4.5 \times 10^{-5}\ [kg * m^2]$
Viscous Friction	$b = 10^{-3}\ \left[\frac{N * m * s}{rad}\right]$
Torque Constant	$K_t = 0.5\ \left[\frac{N * m}{A}\right]$
Sampling Period	$T = 0.001\ [s]$

With the plant and corresponding hardware parameters chosen, the system's equations of motion then must be defined. Assuming a current-controlled drive where the control effort $u(t)$ represents that applied current, the governing differential equation that maps the input to the output (angular position) is given by:

$$J\frac{d^2\theta(t)}{dt^2} + B\frac{d\theta(t)}{dt} = K_t u(t)$$

Algebraically manipulating the equation to isolate the angular acceleration results in:

$$\frac{d^2\theta(t)}{dt^2} = \frac{K_t}{J}\,u(t) - \frac{B}{J}\frac{d\theta(t)}{dt}$$

As is standard practice for controller integration, the differential equation was then translated into a continuous state-space representation. With the states defined as angular position and velocity, the following state-space matrices were defined as follows:

$$A_c = \begin{bmatrix} 0 & 1 \\ 0 & -b/J \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ K_t/J \end{bmatrix}$$

$$C_c = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad D_c = 0$$

With the continuous system defined, the state-space representation was then discretized. This is necessary as, in modern manufacturing applications, systems involve digital controls with discrete sampling, none are truly continuous. Using the sampling period T defined in Table 1, the continuous matrices were converted into the discrete-time domain using a Zero-Order Hold (ZOH). The ZOH method was employed as it can accurately model the physical behvior of a digital-to-analog converter by maintaining a constant control signal between each sample interval. This led to the standard discrete-time state-space equation along with the following discretized matrices (note that the numerical values are substituted into the matrices now):

$$x_{k+1} = A_d x_k + B_d u_k$$

$$A_d = \begin{bmatrix} 1.0000 & 0.0010 \\ 0 & 0.9780 \end{bmatrix}, \quad B_d = \begin{bmatrix} 0.0055 \\ 10.9886 \end{bmatrix}$$

Finally, the system model needs a reference trajectory to benchmark the controller performances. To keep things simple, the study chose a 1 Hz continuous sine wave, which can represent a sort of oscillatory "scanning" operation within a manufacturing environment, and will provide a more rigorous test of phase lag and dynamic tracking capabilities than something such as a step response would. This reference trajectory (y_d) can be seen below in Figure 1.

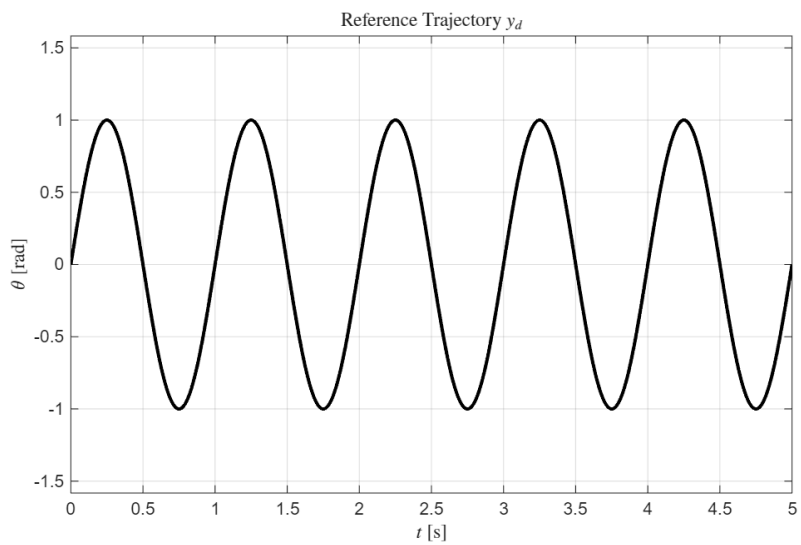

Figure 1: Reference Trajectory

E. CONTROL SYSTEM DESIGN

With the discretized plant defined above in Section D, the study then designed three discrete-time control architectures to be evaluated on the system.

Baseline PID Controller Design

To properly examine the performance of the two more sophisticated control methods, a PID controller was implemented to establish a baseline of uncompensated tracking error and measurement noise. The standard discrete-time PID transfer function in the z-domain is given by:

$$C(z) = \frac{U(z)}{E(z)} = K_p + K_i \frac{T}{1 - z^{-1}} + K_d \frac{1 - z^{-1}}{T}$$

$$K_p = 1.5 \quad K_i = 1 \quad K_d = 0.1$$

The controller gains above were heuristically tuned to minimize tracking error without any extreme control effort. Futhermore, to ensure the causality of the transfer function for a discrete-time simulation, a low-pass filter with a time constant equal to the sampling period ($T_f = T$) was applied to the derivative term.

ILC Implementation

To learn and eliminate the repeated error that the PID system sees, an ILC algorithm was implemented as a feedforward layer on top of that closed-loop PID architecture. The ILC update law utilizes its standard equation:

$$u_{k+1}(t) = Q(q)[u_k(t) + L(q)e_k(t)]$$

Where L is the learning matrix and Q is the zero-phase low-pass filter. The L matrix is constructed using the "lifted system" approach touched on in [1]. This method first defines the transfer function from the feedforward input to the output, followed by calculating the discrete impulse response of this transfer function. The coefficients of the impulse response (called Markov parameters) are then used to construct the lower-triangular Toeplitz matrix (G). The inverse of this matrix ($L = G^{-1}$) is then used to determine the optimal feedforward effort required to eliminate this error.

The robustness filter, Q, as discussed in [3], is then implemented using a cutoff frequency of 70 Hz. This keeps the ILC from learning and amplifying high-frequency measurement noise and instead keeps it focused around the system's actual repeatable dynamics.

MRAC Implementation

To try to better track the system without prior learning, the DEMCS algorithm described in [4] was employed to force the plant to mimic an ideal reference model in real-time. First, the reference model had to be defined, with a desired settling time and damping ratio chosen to force the system to track the 1 Hz reference sine wave. In this application, a settling time of 0.01 s and damping ratio of 0.8 was chosen. The quick settling time forces the DEMCS implementation to very quickly track the reference trajectory, and the damping ratio allows for quick transient response with little overshoot. With these two parameters defined, the natural frequency could be defined and along with the corresponding reference model:

$$\omega_n = \frac{4}{\zeta t_s}$$

$$A_{mc} = \begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}, \qquad B_{mc} = \begin{bmatrix} 0 \\ \omega_n^2 \end{bmatrix}, \qquad C_{mc} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad D_{mc} = 0$$

These continuous-time matrices were then converted into the discrete-time domain the same way the system plant was defined. The standard update laws for the adaptive gains (proportional and integral weights) at each time step were then used:

$$K_i(k) = K_i(k-1) + \alpha v(k) z^T(k-1)$$

$$K_p(k) = \beta v(k) z^T(k-1)$$

$$K(k) = K_i(k) + K_p(k)$$

Where $v(k)$ is the filtered tracking error derived from the Lyapunov Equation, $z(k)$ is the augmented state vector, and α, β are the integral and proportional learning rates. A learning rate of 0.2 was also assigned to the MRAC implementation. This learning rate was determined based on the tradeoff observed during the architecture's tuning: a high learning rate forces tracking but also needs to be low enough to avoid numerical divergence as a result of the sensor noise. As discussed in [4], the positive definite matrix for the Lyapunov Equation Q was defined as an identity matrix scaled by the sampling period to again prevent computational instability.

F. RESULTS & DISCUSSION

As discussed in Section E, the first result to examine would be the baseline PID's tracking performance. With heuristic tuning, the PID performs fairly well, but does see repeated error, as can be seen in Figure 2.

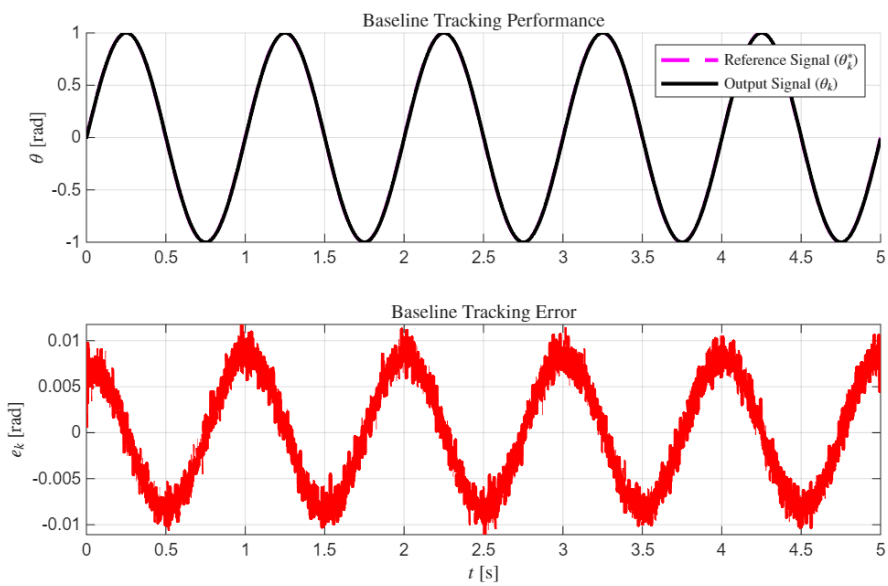

Figure 2: PID Baseline Tracking Performance

To eliminate the repeated tracking error seen in Figure 2, the ILC was implemented as a feedforward layer and ran for 30 trials, the same number of trials found in [3]. A robustness filter cutoff frequency of 70 Hz and learning rate of 0.4 were chosen. This specific learning rate was chosen to exemplify the reduction in error throughout the successive trials. However, a higher learning rate could be employed to achieve faster convergence while maintaining a control input. The resulting performance can be seen below in Figure 3.

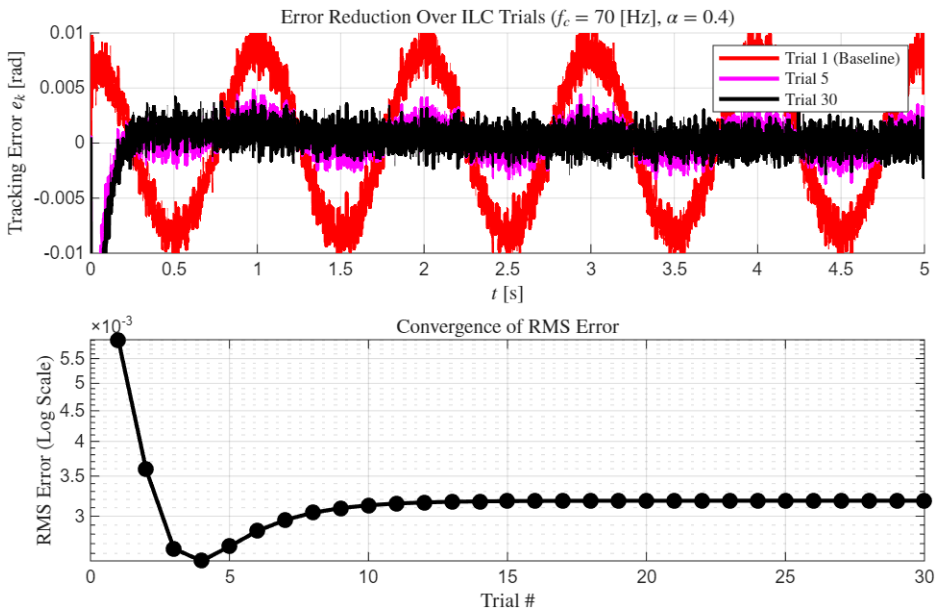

Figure 3: ILC Error Reduction

As can be seen, the ILC successfully reduces tracking error over the 30 trials. As can be seen in the top subplot, Trial 1 represents the PID's performance, followed by Trial 5, where the error is already significantly reduced with still some residual oscillatory error. Finally, Trial 30 demonstrates very little error throughout the whole operation. The bottom subplot presents the rapid convergence of the RMS error, which eventually converges to a value on the order of 10^{-3}.

Finally, the discrete-time MRAC was implemented to track the sinusoidal reference signal. Using the desired system model defined in Section E, the DEMCS algorithm was employed, resulting in the following tracking performance:

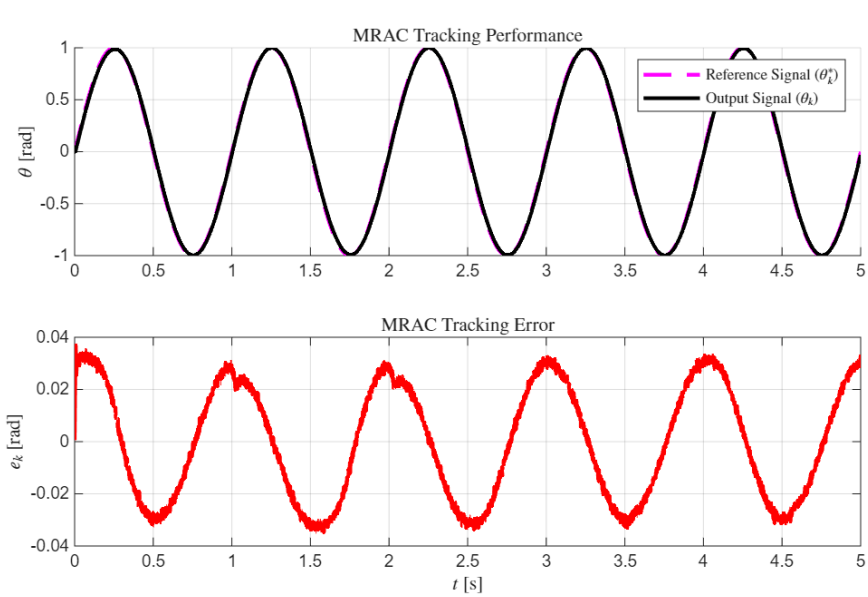

Figure 4: MRAC Performance

As can be seen, the MRAC actually has slightly larger peak tracking error than the baseline PID. This persistent error is a direct consequence of phase lag. Because the MRAC algorithm is restricted by real-time causality and a bounded learning rate to avoid numerical divergence, it is continuously reacting to the trajectory than anticipating it.

To compile the findings thus far, the ILC's 30th rial was compared against the real-time MRAC. Figure 5 presents these performances head-to-head, demonstrating the time-domain tracking error, control effort, and RMS error convergence.

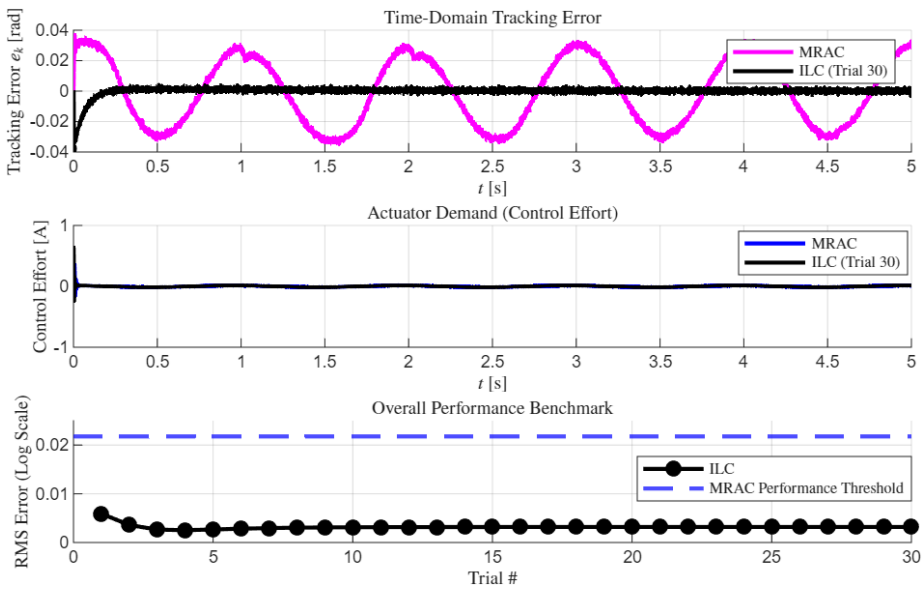

Figure 5: ILC vs MRAC Comparison

As seen in the subplots, while both controllers call a similar total amount of control effort, the ILC vastly outperforms the MRAC in terms of overall tracking. This disparity is a direct result of the principle of causality. The ILC takes advantage of a non-causal lifted-system matrix from [1] that allows it to anticipate and learn the trajectory with causal filtering [3]. This allows it to utilize minimal control effort while simultaneously effectively eliminating phase lag by Trial 30. On the other hand, the MRAC is strictly causal [4], holding no memory of past iterations, and therefore making it purely reactionary. Because of this, in a repetitive environment such as manufacturing, the ILC's non-causal nature leaves it as the superior control architecture.

G. CONCLUSION

This study conducted a comparative analysis of three distinct discrete-time control architectures - a baseline PID, ILC, and MRAC – evaluated on a simulated NEMA 23 stepper motor. The purpose of the study was to evaluate the performance of the three architectures within a highly constrained manufacturing environment subjected to unmodeled dynamics and sensor noise. The comparative analysis relies on the foundational concepts learned in this class, including discrete-time sampling and z-domain stability techniques that were covered throughout the semester. The results demonstrated that the ILC is by far the optimal control methodology in this environment, as its algorithm takes advantage of the cyclical trajectories and can successfully learn to anticipate the reference signal, eventually driving the residual tracking error to a cycle of minimal error. Furthermore, while the MRAC demonstrated robust real-time tracking characteristics without any prior learning or "history", its causality resulted in consistent phase lag. With more time, the study could be expanded upon by moving from a purely software-based architecture to implementation on real hardware, integrated with a controller such as an Arduino. Additionally, future additions to the study could involve an MRAC+ILC architecture to handle both repeated trajectories as well as unpredictable disturbances within the same structure.

H. REFERENCES

[1] - *Harte, T. J. (2007). Discrete-time model-based Iterative Learning Control: stability, monotonicity and robustness (Doctoral dissertation, University of Sheffield).*

[2] - *Hakvoort, W. B. J., Aarts, R. G. K. M., van Dijk, J., & Jonker, J. B. (2007). Model-based iterative learning control applied to an industrial robot with elasticity. 2007 46th IEEE Conference on Decision and Control, 4185–4190. https://doi.org/10.1109/cdc.2007.4434366*

[3] - *Tsurumoto, K., Ohnishi, W., Hara, Y., Kenjo, A., & Tanaka, Y. (2025). On Analysis of Applying Iterative Learning Control to a Closed-Loop Three-Phase Linear Stepper-Motor. 2025 IEEE International Conference on Mechatronics (ICM), 1–6. https://doi.org/10.1109/icm62621.2025.10934934*

[4] - *Montanaro, U., di Gaeta, A., & Giglio, V. (2014). Robust Discrete-Time MRAC With Minimal Controller Synthesis of an Electronic Throttle Body. IEEE/ASME Transactions on Mechatronics, 19(2), 524–537. https://doi.org/10.1109/tmech.2013.2247614*